```
def mutate(x, mut_rate=0.01):          2
    new_x_ = []
    for char in x.string:
        if random.random() < mut_rate:
            new_x_.extend(random.choices(alphabet,k=1))
        else:
            new_x_.append(char)
    new_x = Individual(''.join(new_x_))
    return new_x
```

- ***1*** Recombines two parent strings into two new offspring
- ***2*** Mutates a string by randomly flipping characters

The preceding recombination function takes two parent strings like “hello there” and “fog world” and randomly recombines them by generating a random integer up to the length of the strings and taking the first piece of parent 1 and the second piece of parent 2 to create an offspring, such as “fog there” and “hello world” if the split happened in the middle. If we have evolved a string that contains part of what we want, like “hello” and another string that contains another part of what we want like “world,” then the recombination process might give us all of what we want.

The mutation process takes a string like “hellb” and, with some small probability (the mutation rate), will replace a character in the string with a random one. For example, if the mutation rate was 20% (0.2), it is probable that at least one of the 5 characters in “hellb” will be mutated to a random character. Hopefully it will be mutated into “hello” if that is the target. The purpose of mutation is to introduce new information (variance) into the population. If all we did was recombine, it is likely that all the individuals in the population would become too similar too quickly, and we wouldn’t find the solution we wanted, because information gets lost each generation if there is no mutation. Note that the mutation rate is critical. If it’s too high, the fittest individuals will lose their fitness by mutation, and if it’s too low, we won’t have enough variance to find the optimal individual. Unfortunately, you have to find the right mutation rate empirically.

In listing 6.3 we define a function that will loop through each individual in a population of strings, compute its fitness score, and associate it with that